

QUESTIONS

Q1

Optimize Student Average Calculation
10 marks

Q2

Abstract Class and Runtime Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter Validation
10 marks

Q5

Copy Constructor and Deep Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor Order
10 marks

Q8

Constructor and Object Initialization
10 marks

Q9

Virtual Destructor and Memory Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q1 Optimize Student Average Calculation

10 marks

```
#include <iostream>
using namespace std;

class Student {
public:
    int marks;
    Student(int m) {
        marks = m;
    }
};

int main() {
    int n;
    cin >> n;

    Student *students = new Student[n]{
        Student(80), Student(90), Student(70), Student(60), Student(100)
    };

    for (int i = 0; i < n; i++) {
        int sum = 0;
        for (int j = 0; j <= i; j++) {
            sum += students[j].marks;
        }
        cout << (double)sum / (i + 1) << endl;
    }

    delete[] students;
    return 0;
}
```

Your Code

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Student {
6 public:
7     int marks;
8     Student() : marks(0) {}
9     Student(int m) {
10         marks = m;
11     }
12 };
13
14 int main() {
15     int n;
16     if (!cin >> n) return 0;
17
18     Student *students = new Student[n];
19     int initialMarks[] = {80, 90, 70, 60, 100};
20
21     for (int i = 0; i < n && i < 5; i++) {
22         students[i] = Student(initialMarks[i]);
23     }
24
25     int dynamicSum = 0;
26     for (int i = 0; i < n; i++) {
27         dynamicSum += students[i].marks;
28         cout << (double)dynamicSum / (i + 1) << endl;
29     }
30
31     delete[] students;
32     return 0;
33 }
34
```

Input

5

Output

Q1

Optimize Student Average
Calculation
10 marks

Q2

Abstract Class and Runtime
Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter
Validation
10 marks

Q5

Copy Constructor and Deep
Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor
Order
10 marks

Q8

Constructor and Object
Initialization
10 marks

Q9

Virtual Destructor and Memory
Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

```
#include <iostream>
using namespace std;

class Employee {
public:
    void calculateSalary() {
        cout << "Employee Salary" << endl;
    }
};

class FullTimeEmployee : public Employee {
    double salary;
public:
    FullTimeEmployee(double s) {
        salary = s;
    }

    void calculateSalary() {
        cout << salary << endl;
    }
};

class PartTimeEmployee : public Employee {
    int hours;
    double rate;
public:
    PartTimeEmployee(int h, double r) {
        hours = h;
        rate = r;
    }

    void calculateSalary() {
        cout << hours * rate << endl;
    }
};

int main() {
    int hours;
    double salary, rate;

    cin >> salary;
    cin >> hours >> rate;

    Employee *e1 = new FullTimeEmployee(salary);
    Employee *e2 = new PartTimeEmployee(hours, rate);

    e1->calculateSalary();
    e2->calculateSalary();

    delete e1;
    delete e2;
    return 0;
}
```

Your Code

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Employee {
6 public:
7     virtual ~Employee() {}
8     virtual void calculateSalary() = 0;
9 };
10
11 class FullTimeEmployee : public Employee {
12     double salary;
13 public:
14     FullTimeEmployee(double s) {
15         salary = s;
16     }
17     void calculateSalary() override {
18         cout << salary << endl;
19     }
20 };
21
22 class PartTimeEmployee : public Employee {
23     int hours;
24     double rate;
25 public:
26     PartTimeEmployee(int h, double r) {
27         hours = h;
28         rate = r;
```

View Only

QUESTIONS

Q1

Optimize Student Average
Calculation
10 marks

Q2

Abstract Class and Runtime
Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter
Validation
10 marks

Q5

Copy Constructor and Deep
Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor
Order
10 marks

Q8

Constructor and Object
Initialization
10 marks

Q9

Virtual Destructor and Memory
Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q3 Static Data Member

10 marks

```
#include <iostream>
using namespace std;

class Employee {
    int count;
public:
    Employee() {
        count++;
    }

    void displayCount() {
        cout << count << endl;
    }
};

int main() {
    Employee e1;
    Employee e2;
    Employee e3;
    e3.displayCount();
    return 0;
}
```

Your Code

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Employee {
6 public:
7     static int count;
8
9     Employee() {
10         count++;
11     }
12
13     void displayCount() {
14         cout << count << endl;
15     }
16 };
17
18 int Employee::count = 0;
19
20 int main() {
21     Employee e1;
22     Employee e2;
23     Employee e3;
24     e3.displayCount();
25     return 0;
26 }
```

Input

Output

Q1

Optimize Student Average Calculation
10 marks

Q2

Abstract Class and Runtime Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter Validation
10 marks

Q5

Copy Constructor and Deep Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor Order
10 marks

Q8

Constructor and Object Initialization
10 marks

Q9

Virtual Destructor and Memory Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

```
#include <iostream>
using namespace std;

class BankAccount {
public:
    double balance;

    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {
        balance -= amount;
    }

    void display() {
        cout << balance << endl;
    }
};

int main() {
    BankAccount account;
    double initial, deposit, withdraw;
    cin >> initial >> deposit >> withdraw;
    account.balance = initial;
    account.deposit(deposit);
    account.withdraw(withdraw);
    account.display();
    return 0;
}
```

Your Code

```
1 #include <iostream>
2
3 using namespace std;
4
5 class BankAccount {
6 private:
7     double balance;
8
9 public:
10    void setBalance(double initial) {
11        if (initial >= 0) {
12            balance = initial;
13        } else {
14            balance = 0;
15        }
16    }
17
18    void deposit(double amount) {
19        if (amount > 0) {
20            balance += amount;
21        }
22    }
23
24    void withdraw(double amount) {
25        if (amount > 0 && amount <= balance) {
26            balance -= amount;
27        }
28    }
29
30    void display() {
31        cout << balance << endl;
32    }
33 };
34
35 int main() {
36     BankAccount account;
37     double initial, depositAmount, withdrawAmount;
38
39     if (!(cin >> initial >> depositAmount >> withdrawAmount)) return 0;
40
41     account.setBalance(initial);
42     account.deposit(depositAmount);
43     account.withdraw(withdrawAmount);
44     account.display();
45
46     return 0;
47 }
```

Input

1000 500 200

QUESTIONS

Q1

Optimize Student Average Calculation
10 marks

Q2

Abstract Class and Runtime Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter Validation
10 marks

Q5

Copy Constructor and Deep Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor Order
10 marks

Q8

Constructor and Object Initialization
10 marks

Q9

Virtual Destructor and Memory Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q5 Copy Constructor and Deep Copy

10 marks

```
#include <iostream>
using namespace std;

class Number {
    int *value;
public:
    Number(int v) {
        value = new int(v);
    }

    void display() {
        cout << *value << endl;
    }
};

int main() {
    Number n1(100);
    Number n2 = n1;
    n1.display();
    n2.display();
    return 0;
}
```

Your Code

```
1 #include <iostream>
2
3 using namespace std;
4
5 class Number {
6     int *value;
7
8 public:
9     Number(int v) {
10         value = new int(v);
11     }
12
13     Number(const Number &n) {
14         value = new int(*n.value);
15     }
16
17     ~Number() {
18         delete value;
19     }
20
21     void display() {
22         cout << *value << endl;
23     }
24 };
25
26 int main() {
27     Number n1(100);
28     Number n2 = n1;
29     n1.display();
30     n2.display();
31     return 0;
32 }
```

Input

stdin input

Output

QUESTIONS

Q1

Optimize Student Average Calculation
10 marks

Q2

Abstract Class and Runtime Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter Validation
10 marks

Q5

Copy Constructor and Deep Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor Order
10 marks

Q8

Constructor and Object Initialization
10 marks

Q9

Virtual Destructor and Memory Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q6 Operator Overloading

10 marks

```
#include <iostream>
using namespace std;

class Complex {
    int real, imag;
public:
    Complex(int r = 0, int i = 0) {
        real = r;
        imag = i;
    }

    Complex operator+(Complex c) {
        Complex temp;
        temp.real = real;
        temp.imag = imag;
        return temp;
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    int r1, i1, r2, i2;
    cin >> r1 >> i1;
    cin >> r2 >> i2;

    Complex c1(r1, i1);
    Complex c2(r2, i2);
    Complex c3 = c1 + c2;
    c3.display();
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Complex {
5     int real, imag;
6
7 public:
8     Complex(int r = 0, int i = 0) {
9         real = r;
10        imag = i;
11    }
12
13    Complex operator+(Complex c) {
14        Complex temp;
15        temp.real = real + c.real;
16        temp.imag = imag + c.imag;
17        return temp;
18    }
19
20    void display() {
21        cout << real << " + " << imag << "i" << endl;
22    }
23 };
24
25 int main() {
26     int r1, i1, r2, i2;
27
28     cout << "Enter real and imaginary parts for first number: ";
29     cin >> r1 >> i1;
30     cout << "Enter real and imaginary parts for second number: ";
31     cin >> r2 >> i2;
32
33     Complex c1(r1, i1);
34     Complex c2(r2, i2);
35
36     Complex c3 = c1 + c2;
37
38     cout << "Result: ";
39     c3.display();
40
41     return 0;
42 }
```

Input

QUESTIONS

Q1

Optimize Student Average
Calculation
10 marks

Q2

Abstract Class and Runtime
Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter
Validation
10 marks

Q5

Copy Constructor and Deep
Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor
Order
10 marks

Q8

Constructor and Object
Initialization
10 marks

Q9

Virtual Destructor and Memory
Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q7 Inheritance and Constructor Order

10 marks

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle() {
        cout << "Vehicle Constructor" << endl;
    }
};

class Car : Vehicle {
public:
    Car() {
        cout << "Car Constructor" << endl;
    }
};

int main() {
    Car c;
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Vehicle {
5 public:
6     Vehicle() {
7         cout << "Vehicle Constructor" << endl;
8     }
9 };
10
11 class Car : public Vehicle {
12 public:
13     Car() {
14         cout << "Car Constructor" << endl;
15     }
16 };
17
18 int main() {
19     Car c;
20     return 0;
21 }
```

Input

stdin input

Output



QUESTIONS

Q1

Optimize Student Average Calculation
10 marks

Q2

Abstract Class and Runtime Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter Validation
10 marks

Q5

Copy Constructor and Deep Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor Order
10 marks

Q8

Constructor and Object Initialization
10 marks

Q9

Virtual Destructor and Memory Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q8 Constructor and Object Initialization

10 marks

```
#include <iostream>
using namespace std;

class Student {
    string name;
    int marks;
public:
    Student(string n, int m) {
        n = name;
        m = marks;
    }
    void display() {
        cout << name << " " << marks << endl;
    }
};

int main() {
    string name;
    int marks;
    cin >> name >> marks;
    Student s(name, marks);
    s.display();
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Student {
5     string name;
6     int marks;
7
8 public:
9     Student(string n, int m) {
10         name = n;
11         marks = m;
12     }
13
14     void display() {
15         cout << name << " " << marks << endl;
16     }
17 };
18
19 int main() {
20     string name;
21     int marks;
22
23     cin >> name >> marks;
24
25     Student s(name, marks);
26     s.display();
27
28     return 0;
29 }
```

Input

Output

QUESTIONS

Q1

Optimize Student Average Calculation
10 marks

Q2

Abstract Class and Runtime Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter Validation
10 marks

Q5

Copy Constructor and Deep Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor Order
10 marks

Q8

Constructor and Object Initialization
10 marks

Q9

Virtual Destructor and Memory Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

Q9 Virtual Destructor and Memory Leak

10 marks

```
#include <iostream>
using namespace std;

class Parent {
public:
    ~Parent() {
        cout << "Parent Destructor" << endl;
    }
};

class Child : public Parent {
public:
    ~Child() {
        cout << "Child Destructor" << endl;
    }
};

int main() {
    Parent *p = new Child();
    delete p;
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Parent {
5 public:
6     virtual ~Parent() {
7         cout << "Parent Destructor" << endl;
8     }
9 };
10
11 class Child : public Parent {
12 public:
13     ~Child() {
14         cout << "Child Destructor" << endl;
15     }
16 };
17
18 int main() {
19     Parent *p = new Child();
20     delete p;
21     return 0;
22 }
23
```

Input

stdin input

Output

Q1

Optimize Student Average
Calculation
10 marks

Q2

Abstract Class and Runtime
Polymorphism
10 marks

Q3

Static Data Member
10 marks

Q4

Encapsulation and Setter
Validation
10 marks

Q5

Copy Constructor and Deep
Copy
10 marks

Q6

Operator Overloading
10 marks

Q7

Inheritance and Constructor
Order
10 marks

Q8

Constructor and Object
Initialization
10 marks

Q9

Virtual Destructor and Memory
Leak
10 marks

Q10

Runtime Polymorphism
10 marks

10/10 answered

```
#include <iostream>
using namespace std;

class Shape {
public:
    void area() {
        cout << "Shape Area" << endl;
    }
};

class Rectangle : public Shape {
public:
    void area() {
        int l, b;
        cin >> l >> b;
        cout << l * b << endl;
    }
};

class Circle : public Shape {
public:
    void area() {
        int r;
        cin >> r;
        cout << 3.14 * r * r << endl;
    }
};

int main() {
    Shape *s;
    Rectangle r;
    Circle c;

    s = &r;
    s->area();

    s = &c;
    s->area();
    return 0;
}
```

Your Code

```
1 #include <iostream>
2 using namespace std;
3
4 class Shape {
5 public:
6     void area() {
7         cout << "Shape Area" << endl;
8     }
9 };
10
11 class Rectangle : public Shape {
12 public:
13     void area() {
14         int l, b;
15         cin >> l >> b;
16         cout << l * b << endl;
17     }
18 };
19
20 class Circle : public Shape {
21 public:
22     void area() {
23         int r;
24         cin >> r;
25         cout << 3.14 * r * r << endl;
26     }
27 };
28
29 int main() {
30     Shape s;
31     Rectangle r;
32     Circle c;
33
34     s.area();
35     r.area();
36     c.area();
37
38     return 0;
39 }
```